Revised Implementation Plan: Avenir Payment Foundations

Oct 15, 2026 · @Shantha

0. Objective, boundary and what this plan supersedes

Objective, as you set it on 15 October: a reliable, professionally structured Avenir system, so that accurate data can be entered again from a starting date you will confirm. The software is the work; historical reconciliation, old duplicate amounts and corrections to old accounting records are set aside for this phase and must not block it. The existing data is development material: it reproduces faults and shows workflows, and nothing in it (amounts, statuses, behaviour) is taken as correct. Your monthly bank checks found no duplicate bank payments; that stands as recorded. **[Your instruction]**

What this plan supersedes. The consolidated plan and the Final Next-Step Package of 15 October remain the record of the read-only investigation and of Ravi's pack for voucher 7134 (prepared, not sent). Their Stream H (historical reconciliation) is parked, not cancelled. Their Stream S (software faults) is replaced by sections 3–5 here, widened from the authorisation guard to the whole payment workflow. The structural compatibility register's open items stay open where this plan says so. **[Proposal]**

Boundary, applied throughout. Work is on the isolated localhost copy only (section 2). The staff's live system is not accessed or changed. The development copy at C:\wamp64\www\avenir and its database are not changed by this plan; they are read for evidence. No existing record is deleted, reset or overwritten: test records are added, labelled TEST, and listed. Staff assignments and permissions are yours; the plan names what each change needs, never grants it. Approvals: each build, each install on the test copy beyond this one, each write test set and the clean database each need your word; this plan asks for them in section 7.

Labels. [Observed] with its source; [Deduced]; [Proposal]; [Assumption] where not verified; [Blocker] where something cannot proceed without you or IT.

1. Already proven (evidence on record; not to be repeated)

Proven	Evidence	Source
The code has four authorisation paths (V1 Voucher Print / Bank Payment Check, V2/V3 the desk's Paid which calls V1 internally, V4 bulk hotel accept, V5 advance transfer); only the desk refuses a second authorisation; V1 refuses only a voucher marked rej	code read, file:line references	follow-up report section 6 (F-1 to F-7)
The ledger tables account_runing and account_runing_2024 are MyISAM with no primary key and a non-unique ID; voucher has a unique index on Vnum; v_inv has no index on V_tkn	M2, 15 Oct	evidence folder followup-run\06
A second authorisation of one voucher posts its ledger again: voucher 7134 carries two identical journals (26061, 26075; 13 Jul 2026); every other sup = 0 authorised voucher carries one	G1c, G2r, G3, G4	followup-run\08–14; Final package section 12
Request statuses can be changed from any state by seven routes without a status check or lock (approval, bulk approval, create_bulk_voucher, save_payment, completion, lunch_pay, cancellation); bulk rejection sets a bulk-type request to 6 (approved); a bulk lunch approval sets every other request of the hotel account to 6	code read	A2 section 4a; R0 R14; r3.5 12b
Dashboards derive “paid” from tour flags that disagree with stored status (266 at status 4 show flag 1; 21 at status 2 show flag 0)	R0 data	R0 R16

Proven	Evidence	Source
The voucher-number counter is read-then-written without a lock (invoice_no), so two writers can take one number; lines are written before the voucher	code read	follow-up report D9; InvoiceNumberModel
Bank Payment Check's mark_finalized checks neither Aflag nor finalized nor separation	code read	follow-up report D11
Driver advances: three creation routes check nothing against each other; rejecting a Transport Advance request resets every request row of the driver on the tour, paid ones included; the top-up repays posted expenses	code read	F-2, F-3, D3
No creation route has a duplicate check; CSRF is sent by pages but checked nowhere	code read	D8; F-7
The isolated test copy avenir_rv exists, forces every connection to avenir_restore_verify_20261011, shows a red bar, has its own cookies and TEST logins	README-avenir_rv.txt; Database.php override; T-rv read (15 Oct 12:58 UTC)	section 2
MySQL 8.4.7 applies MAX_EXECUTION_TIME hints; performance_schema is on	H0, H1 × 10	followup-run\02–21

Not proven, and treated as unverified: that any existing posting rule is correct; that the current status model (0, 1, 2, 3, 4, 5, 6) is complete or right; that the existing audit (tpd_audit, from 11 October) is tamper-proof; that the application's reports agree with the underlying records. Each of these is tested, not assumed, in section 5.

2. The isolated workspace and the backup and restore arrangements, with the unresolved restrictions named

The isolated workspace: C:\wamp64\www\avenir_rv [Observed, 15 Oct 12:5x UTC]

Item	Fact	How confirmed
Code	a copy of the development code of 11 October with the later kits; 876 PHP files in app, 1,726 files in app\ and public\	file count in the linked folder
Address	http://localhost/avenir_rv/ (this laptop only); every page carries the red bar “ISOLATED TEST COPY (avenir_rv) … dummy records only — not the development system”	App.php baseURL; the page itself
Database	avenir_restore_verify_20261011 only: app\Config\Database.php forces the groups default, secondary and secondDatabase to it in code, so .env cannot point it elsewhere	Database.php lines 115–135; T-rv: DATABASE() = avenir_restore_verify_20261011, MySQL 8.4.7, port 3306
Sessions	its own cookies (avenir_rv_session, avenir_rv_csrf); signing in here never signs anyone out of development	README-avenir_rv.txt
External services	no live keys in .env (no SMS, WhatsApp, AI or e-mail credentials)	README; .env holds CI_ENVIRONMENT only
Logins	TEST users: TestTourRes, TestHotelRes, TestTransport, TestFinApprover, TestBankAuth, TestAccounts, TestOperations (user ids 116–122) plus test, test2	user_acc_tbl, ids and names only
Data	the development data of 11 October (voucher 7,975; v_inv 8,178; account_runing 11,390 before today's TEST rows) with dummy records: tour 990001 / AH TEST-9901, drivers 6-9-11-99901/99902, ledger ids 99990001–02	counts 15 Oct; README

Backups taken on 15 October before any change, and how each is verified [Observed]

Backup	Where	Verification
Files: app\ of the test copy (1,001 files)	C:\wamp64\www\avenir_rv\writable\app_backups\app-before-payguard-20261015-125912.tar.gz (SHA-256 b291e70c…) with app-before-payguard-20261015-125912.sha256 (1,001 lines)	the archive lists 1,001 files; the manifest has 1,001 entries
Tables the fix and tests can write: voucher, v_inv, account_runing, invoice_no, tour_quotations, consignee, driver_allocation, tpd_audit	copies inside the test database, zz_bak_20261015_1259_<table>	row counts equal (7,975 / 8,178 / 11,390 / 1 / 11,089 / 1,869 / 630 / 9,719) and CHECKSUM TABLE EXTENDED equal for every pair
Restore	files: unpack the archive over app\ and compare with the manifest; tables: `RENAME TABLE` the copy back over the table, or `DELETE` the TEST rows by their listed ids (section 4)	both rehearsable on the test copy before any use

Reusing existing permissions. The folder C:\wamp64\www\avenir_rv is already connected to this session (read/write), the test database is reached through phpMyAdmin with your sign-in, and the TEST logins exist: nothing new was granted and nothing is asked for. Deletion in the connected folder is off (files I replace are moved to writable_to_delete, never deleted). **[Observed]**

Unresolved restrictions, stated precisely [Blocker where marked]

1. **Standing of avenir_rv under the structural restriction.** It lies inside C:\wamp64\www but outside C:\wamp64\www\avenir. Your instruction of 15 October to “prepare and test the fix on the isolated localhost test copy” is read as naming it; if the restriction excludes it, say so and the work stops there. *[Assumption, stated]*
2. **Database backup method.** The table copies above live inside the same MySQL server and database as the test copy. A dump to a file would need either MySQL's own file output (secure_file_priv not yet read: S1 of the Final package) or a client on this laptop that the session's shell cannot reach (it cannot connect to your localhost MySQL). A copy outside the server is therefore not on record. *[Blocker only for a disaster that loses the MySQL data directory; not for the tests]*

3. **Verified restore on a separate server** (r3.5 17b) is not possible inside the structure without a second MySQL instance (option T2); for the test copy it is not required, for development it remains the open item U-5 of the register.
4. **Two different users at the same moment** need two signed-in browsers; the TEST passwords are not in any project fixture, so only you can sign them in (section 4 records what was possible).
5. **No PHP runs in the session's shell on this laptop**, so syntax is checked by a one-off page on the test copy (used on 15 Oct and moved out of public\ afterwards) and behaviour by the application itself; PHPUnit suites in tests\ cannot be run from here. *[Limitation]*

3. What needs repair, stage by stage

Order of work: each item is built on the test copy, tested against section 5, packaged with before-copies and checksums, and installed on development only with your approval after a verified backup. Retain-or-refactor is decided per item from the evidence: where the existing code is sound apart from a missing check, the check is added (R-1, R-2, R-4); where the mechanism itself is unsafe on MyISAM, the step is rebuilt around a lock, a claim and an undo (R-1, R-5); a rewrite of the whole chain is not assumed. **[Proposal]**

#	Stage	Fault on record	Repair	Status
R-1	Ledger posting at authorisation (V1, desk Paid via V1, V4, V5)	a second authorisation posts again (D1); V4 posts k times (D2); V5 posts with no checks (D10); a failure part-way leaves rows with no undo (MyISAM)	**the authorisation guard**: named lock per voucher, re-read under the lock, InnoDB claim row, undo of every row written on failure, audit of every outcome; V4 withdrawn; V5 guarded with an operation key	**built on the test copy 15 Oct; tests in section 4**

#	Stage	Fault on record	Repair	Status
R-2	Request status changes (P1, P2, P3, P4, P5, P6, P7, Q1, Q2)	no status check or lock on seven routes; bulk rejection → 6; bulk lunch approval flips other requests; issue_voucher can leave a request at 1	one status engine: every change goes through a compare-and-set (`UPDATE … WHERE status = <expected>` and affected rows = 1) under a per-request named lock; allowed transitions written once (1→4/6, 1→5, 1→3, 4/6→2 with a finalized voucher); bulk actions act only on the selected rows; a transition row per change	next after R-1
R-3	Paid items returning to queues	Q1 cancels any status; transport rejection resets paid rows; lunch approval flips completed requests; dashboards read tour flags	R-2′s transitions make these impossible; the transport reset and the lunch side-effect are removed; panels read stored status, flags become display-only	with R-2
R-4	Voucher creation and numbering	read-then-write counter; lines written before the voucher; no duplicate key on creation routes (D8, D9)	number taken under a named lock with `UPDATE invoice_no SET V_Num = LAST_INSERT_ID(V_Num + 1)` ; voucher row written before its lines, lines carry the number; a business key per creation route (request id, bill id, settlement id) in an InnoDB key table; double submit refused	after R-2

#	Stage	Fault on record	Repair	Status
R-5	Bank-payment recording (Bank Payment Check mark_finalized; desk Paid)	finalises an unposted, cancelled or already finalized voucher; overwrites a bank date (D11); a desk Paid failure after finalized leaves charges unposted (F-4)	state checks (Aflag = 1, finalized = 0) under the voucher lock; bank date written once; the desk′s post-finalize steps made resumable with their own claim	after R-4
R-6	Completion (P6, P7)	completes any request regardless of status or payment	completion only from 4/6 with a finalized voucher whose amount covers the request; part of R-2′s engine	with R-2
R-7	Driver advances and expenses	three uncoordinated creation routes; top-up repays posted expenses (D3); two accepts post twice (D5); settlement double charge (D7)	one creation path with the R-4 key; accept under a lock, one posting per expense; top-up limited to the unpaid balance; the settlement posting rule referred to the accountant before change	after R-4
R-8	Permissions and audit	routes pass on menu paths alone; CSRF unchecked; tpd_audit has no protection against update or delete; legacy screens keep no audit	action-level checks per payment route (who may approve, authorise, finalize, complete), the CSRF filter on payment routes, append-only triggers on the audit table, audit calls in every payment route (R-1 already writes them)	alongside R-2 to R-6

#	Stage	Fault on record	Repair	Status
R-9	Reporting	reports and dashboards computed from flags and from ledger rows that may be duplicated	each report rebuilt from stored status and from one posting per voucher, with a reconciliation check (report total = sum of records) in the test suite	last
R-10	Storage engine	the eleven payment tables of 10a Group A are MyISAM: no transaction, no rollback, duplicate journal ids possible	two paths, your decision: keep MyISAM and rely on lock + claim + undo (R-1′s pattern) for every writer; or convert the eleven tables of 10a Group A to InnoDB (package P-2, section 14) and add real transactions. Either way: a unique key on the journal id is added so two journals can never share an id	decision needed (section 7)

Integrity demonstration required for every item (section 5): a failure injected after the first write must leave the tables exactly as before the attempt (counts and checksums equal), the status unchanged, and an audit row saying what was undone.

Added to this list on 15 October from the inventory of section 10a-2 (none touched before the engine decision): the dashboard2 voucher_auth confirmation routes (D-05); the voucher cancellation route with its duplicate-row reversal (D-06); the creditor-invoice complete_payment and expenses_delete routes (D-07, D-08); the driver settlement, driver expenses and driver fund-return routes that create vouchers at Aflag 0 or post ledger pairs without a lock, a duplicate check or a transaction (D-08). Each is a writer of the payment tables and receives the same posting engine (one business key per operation) when the remaining routes are rebuilt. [Observed: code; Proposal]

4. Repair item 1, built on the test copy: the authorisation guard (tf-payguard-r1)

4a. What was changed, where, and why it holds on MyISAM

Target confirmed before any change (section 2): C:\wamp64\www\avenir_rv, database avenir_restore_verify_20261011, MySQL 8.4.7, PHP 8.1.33. Backups taken and verified first (section 2). The live staff system and the development copy were not touched. **[Observed, 15 Oct]**

File (under C:\wamp64\www\avenir_rv\app)	Change	Status
Libraries\PayGuard\AuthGuard.php	**new**, 255 lines: claim / complete / fail, snapshot and undo, audit; one named lock per voucher number (GET_LOCK, 10 s wait); re-read of the voucher and of both ledger tables under the lock; an InnoDB claim table pg_auth_claim with the voucher number as primary key; compensation that deletes the rows written in the attempt by their journal ids and restores voucher status, line status, night flags and transport markers from a snapshot taken under the lock	new, compiles
Config\PayGuard.php	**new**: `testHooks` flag, true only on the test copy (release value false); it enables two request fields, pg_delay_ms and pg_fail, used by the tests below and ignored elsewhere	new, compiles

File (under C:\wamp64\www\avenir_rv\app)	Change	Status
Controllers\Accounts\VoucherPaymentController.php (V1)	`update()`: the claim is taken before any posting when status = 1 and refused with a message (HTTP 409) if the voucher is already authorised, rejected, cancelled, claimed, or already has voucher-payment rows in either ledger table; the whole posting runs in try/catch and any failure calls `AuthGuard::fail()`, which undoes the attempt and answers 500 with what was undone; `acc_array()` records each journal id it writes and throws instead of returning “error” silently; the sup = 2 clearing journal now posts only when authorising (status 1), never on a rejection (the F2 defect); the voucher-number counter is consumed only after a successful claim	edited, compiles
Controllers\Settings\ManualUpdateController.php (V4)	`hotel_voucher_accept()` withdrawn: answers 403 with a message and writes an audit row; the former body kept as a private, unrouted method for reference	edited, compiles
Controllers\NewDashboard\DriverAdvancePaymentController.php (V5)	`transfer_amount()` rebuilt around the same pattern: lock on the original voucher; facts read from the database (the original must be authorised and not rejected; the amount must be above zero and within the voucher's amount; the drivers must differ; a transport-advance ledger entry must exist); an operation key per form (table pg_op, primary key) so a repeated submission is refused; undo of the ledger pair, the amount change, the new line and the new voucher if a later step fails; audit of every outcome	edited, compiles
Views\pages\accounts\voucher_print.php	the page now shows the guard's message in full for 409 and 500 instead of “Error occurred”	edited
Views\pages\dashboard2\transport\transport_advance.php	the transfer form sends an operation key generated when the form opens	edited

Routes covered. V1 accounts/voucher_auth/update (Voucher Print and Bank Payment Check post to it); the Tour Payment Desk's Paid and bulk Paid, which call V1's `update()` internally (TourPaymentDeskController :1458) and therefore pass through the same claim; V4 settings/manual_update/hotel_voucher_accept (withdrawn); V5 dashboard2/transport/driver_advance/transfer_amount. No other current code sets Aflag = 1 (follow-up report section 6). **[Observed: code]**

Why it should hold without transactions — and what is and is not yet evidence. The payment tables are MyISAM, so nothing here relies on rollback. The design: two requests for one voucher are serialised by the named lock whatever their route; the decision to post is taken from a re-read under that lock, not from what the page sent; the claim row's primary key makes a second claim fail in InnoDB even if a lock were lost; a failure after rows were written is undone by deleting exactly those rows (known by journal id) and restoring the snapshot. A disabled button is not relied on anywhere. **Evidence so far, by kind:** *code inspection* — the five routes and their call paths were read in the test copy's files (section 1 sources); *compilation* — all five changed PHP files compile under PHP 8.1.33 (one-off check page, 15 Oct); *expected results* — listed per case in 4c; *observed test results* — **none yet**: no test has run (4c). Until 4c holds observed results, the guard is a built design, not a proven one, and it is **not** put forward as the pattern for the other repairs; your preferred direction (an InnoDB rehearsal first, section 10) is taken up before any extension of lock-claim-undo. **[Design: Deduced; evidence status: Observed]**

4b. Rollback of this repair on the test copy

1. Files: unpack writable\app_backups\app-before-payguard-20261015-125912.tar.gz over app\ (or move the five edited files back from the archive) and compare every file with app-before-payguard-20261015-125912.sha256; the two new files are moved to writable_to_delete.
2. Tables: the TEST rows are removed by their listed ids (section 4c), or voucher, v_inv, account_runing, invoice_no, tour_quotations, consignee, driver_allocation and tpd_audit are restored by `RENAME TABLE` from their zz_bak_20261015_1259_ copies; pg_auth_claim and pg_op are dropped. Counts and checksums must then equal the copies.
3. Nothing on development or live is involved; there is nothing to roll back there.

4c. Test evidence from the test copy

Status at 14:45 UTC, 15 October: the test run started at 14:40 UTC after you signed in to phpMyAdmin and to the test copy. One case has an observed result (G-1); it ran under the test copy's admin login ("super admin", user id 1), not under TestAccounts, which was found from the audit rows and reported to you at 14:42 UTC. The remaining cases wait for the TestAccounts sign-in on tab-4 and, for the two-user cases, TestBankAuth in your own

Chrome. The table below keeps the planned expectation per case; the observed results, with request, response, user, TEST ids, before-and-after data and audit rows, are in section 13 and in the evidence files guard-tests\G-NN.json. Nothing is claimed as observed beyond section 13. [Observed: audit rows 9720–9721, pg_auth_claim 8021]

TEST records created in the test database for these tests (all labelled TEST; listed for removal under 4b):

Record	Ids	Purpose
Vouchers TEST-GUARD-1 to 6 (sup = 0, Bank Transfer, account 6-9-23-11013, Aflag 0), modelled on 7134's structure	voucher ID 8032–8037, Vnum 8021–8026, amounts 1,111.11 … 6,666.66; one v_inv line each (ID 8253–8258)	one voucher per test case below
Voucher TEST-GUARD-TA1 (transport advance, authorised, Aflag 1, 10,000.00, driver 6-9-11-99901, tour 990001) with its ledger journal	voucher ID 8038, Vnum 8027; v_inv ID 8259; account_runing journal 28990 (two legs)	the V5 transfer tests
Counters set to match	invoice_no: V_Num 8027, acc_id 28990	no collision with later creations
Baseline before tests	account_runing 11,392 rows; voucher 7,982; v_inv 8,185; tpd_audit 9,719	the after-counts are compared with these

The test set, with the result each must show (run from the signed-in page, each request's status, message and timing recorded; counts and the audit rows read after each case):

#	Case	Route	Must show
G-1	normal authorisation of TEST-GUARD-1	V1	200; one journal (2 legs) under 8021; Aflag 1; claim "posted"; audit auth_claimed + auth_posted
G-2	double click: two posts fired without waiting, TEST-GUARD-2	V1	one 200 and one 409 with the voucher named; one journal only
G-3	retry after success: TEST-GUARD-1 again	V1	409 "already authorised"; journal count unchanged

#	Case	Route	Must show
G-4	two tabs: the same post from two tabs at once, TEST-GUARD-3	V1	one 200, one 409
G-5	two users at the same moment (or two tabs if only one login is available), TEST-GUARD-4, the first delayed 4 s inside the guarded section	V1	the first 200 after ≈ 4 s; the second waits on the lock, then 409; one journal; both audit rows
G-6	posting failure after the ledger rows were written (test hook), TEST-GUARD-5	V1	500 “failed and was undone”; 0 ledger rows under 8025; Aflag back to 0; claim removed; audit auth_failed_undone naming the deleted journal id; counts equal to before the attempt
G-7	retry after the undone failure, TEST-GUARD-5	V1	200; one journal; the earlier journal id not reused (counter gap recorded)
G-8	reject TEST-GUARD-6, then try to authorise it	V1	reject 200; authorise refused (301/409); no ledger rows
G-9	bulk hotel accept	V4	403 with the withdrawal message; audit row; no count changes
G-10	V5 transfer 3,000 from 99901 to 99902 on TA1 with an operation key	V5	200; one new journal; TA1 amount 7,000; a new authorised voucher for 3,000; audit
G-11	the same operation key again	V5	409 “already submitted”; nothing changes
G-12	transfer above the balance; same driver; a voucher not authorised (TEST-GUARD-6)	V5	409 / 400 / 409 with the reason; nothing changes
G-13	V5 failure after the lines were written (test hook)	V5	500 “undone”; journal, line, voucher and amount restored; counts equal to before
G-14	the Tour Payment Desk’s Paid for a desk voucher	V2	covered by the code path (the desk calls V1’s update() internally); a desk fixture is not in this set — recorded as a limitation unless you want it added

Evidence to be kept for each case: request fields (no credentials), HTTP status, message, sent and received times, the counts and the audit rows read through phpMyAdmin on the test database, and the ledger rows under each TEST voucher number. Where a case cannot be run (two real users), the limitation is written here, not papered over.

5. Test acceptance criteria for the whole workflow

Every test uses controlled TEST records with a written expected result; the existing data is never the expected result. Every run starts from the verified copies (section 2) and ends with the TEST rows listed and the non-TEST tables compared with their copies (count and CHECKSUM TABLE equal). A test passes only when every line of its “must” column holds. **[Proposal]**

Area	Case	Must
Normal processing	request → approve → voucher → authorise → finalize → complete, one TEST obligation	one request row, one voucher, one journal (two legs, balanced, Recpt_No = voucher number), one finalized flag with one bank date, status 2 at the end; the report for the tour shows the one amount
Double click	the same button twice within a second, on authorise, approve, finalize, complete, create	exactly one effect; the second answers with a refusal message naming the voucher or request; one audit row per outcome
Two tabs	the same action from two tabs of one user	as double click
Two users	the same action from two sessions of two users at the same moment (one request delayed inside the guarded section)	one effect; the other refused after the first finishes; both audit rows carry the right user
Retry	the same action again after success, after refusal, after an undone failure	after success: refused; after refusal: refused again; after an undone failure: succeeds once

Area	Case	Must
Interrupted processing	a failure injected after the first write of each multi-write step (ledger rows, status, counter, lines)	tables equal to the state before the attempt (counts and checksums), status unchanged, the counter not consumed or its gap recorded, an audit row “failed, undone”, a message that says nothing was posted
Partial payments	a request paid in two vouchers; a voucher that overpays; a balance request	allocations sum to the request; overpayment refused; completion only when the sum covers the request
Cancellations	cancel at each status (1, 4, 6, 2) and a cancelled voucher	allowed only at 1 with no voucher; all others refused with the status named; a cancelled voucher cannot be authorised or finalized
Alternative routes	every route in the inventory (A2 section 4a) that changes a payment status or posts: V1–V5, P1–P7, Q1–Q2, X1, B1/A1, creation routes, manual tools	each either applies the same engine or is withdrawn with a 403 and an audit row; none can change a status out of sequence
Numbering	20 creations fired together	20 distinct voucher numbers; each line attached to its own voucher; no gap larger than the recorded undone attempts
Permissions	each payment action by each TEST role, and by a user with no role	allowed only for the role named in the plan; refused with a message otherwise; refusal audited; a forged cross-site POST without the token refused
Audit	after every test above	one row per attempt with action, user, voucher or request, outcome and reason; no row can be updated or deleted by the application (trigger refuses)
Reports agree with records	after the suite	every report total equals the sum of the stored records it claims to summarise; no figure comes from a flag

Area	Case	Must
Storage engine	the integrity cases above repeated after the MyISAM-or-InnoDB decision (R-10)	identical results; under InnoDB, additionally no undo code path is needed to reach the same state

Evidence of each run: the request and response (status, message) per case, the before and after counts and checksums, the audit rows, and the TEST record ids, saved in the evidence folder under a dated sub-folder by the capture route you approve.

6. Preparing the clean database for fresh entry (structure and configuration only)

After validation, not before. A separate, empty database is prepared only when sections 3–5 are complete on the test copy and you have approved the repaired code. It carries structure and approved configuration, and nothing else. **[Proposal]**

Carried into the clean database	Not carried
the schema of every table, with the repaired keys and indexes (unique journal id, unique voucher number, business keys, append-only audit triggers) and the storage engine you decide under R-10	any transaction: no vouchers, lines, ledger rows, requests, payments, bank statements or audit rows from the old data
configuration tables you approve: chart of accounts and ledger codes, bank accounts and their mappings, payment types, menus and privileges as you assign them	unverified balances: no opening balance is entered by the system; you and Ravi supply the opening balances separately, and they are entered as dated journals under the normal, guarded routes
master data you confirm: suppliers, hotels, drivers, agents (each list reviewed by you before load; nothing merged or deduplicated automatically)	logins and passwords: accounts are created by you; no TEST login is carried
the counters set to the starting values you choose	historical reports; the old database stays intact and readable beside it

Starting date. Not assumed. The year-end on 31 March and a 1 April start are noted as your intention; the date, the master data and the opening balances are confirmed by you separately, and nothing is imported or reset until then.

Method, when approved. The structure is produced from the repaired test copy by the application's own schema definitions (the installer libraries and app\Database\Sql, extended for the repairs), loaded into a new database name of your choosing on the same MySQL server, and verified: object counts, table engines, keys and triggers equal to the test copy's; row counts zero except the configuration and master tables you approved; a dry run of the full test suite (section 5) on the clean database before any staff use. The old database and its files are preserved unchanged throughout.

7. Genuine blockers and the approvals needed next

Blockers (nothing else stops the work)

#	Blocker	What resolves it
B-1	The TEST sign-in on the test copy: the TEST passwords are in no project fixture, so only you can sign in; without it no write test runs	you sign in as TestAccounts in the browser pane (and, for the two-user case, a second Test* login in your own Chrome)
B-2	The standing of avenir_rv under the structural restriction (section 2, item 1)	one line from you
B-3	The storage-engine decision R-10 (keep MyISAM with lock-claim-undo everywhere, or convert the eleven tables of 10a Group A to InnoDB) — it fixes the shape of R-2, R-4, R-5 and R-7	your decision, after the InnoDB rehearsal of section 10 (package P-2, section 14) if you want it
B-4	The screen and column behind "paid items reappear" and "invoices appear two or three times" (Final package 7d): the repairs R-2/R-3 cover every return point on record, but the specific report you saw cannot be reproduced until it is named	the screen's name as you see it, the column, one example reference
B-5	A database backup outside the MySQL server for the test copy (section 2, item 2) — not needed for the tests, needed before the clean-database step	S1–S3 of the Final package (MySQL file output) or an IT-run dump

Approvals needed next, in order

#	Approval	Unlocks
A-1	The guard (R-1) stays on the test copy as a built repair under test; no acceptance is asked until section 13 holds observed results, and it is not proposed as the pattern for other routes before the InnoDB rehearsal (section 10) is decided	R-2 to R-7 built the same way
A-2	R-10: MyISAM with lock-claim-undo, or InnoDB conversion first	the design of R-2, R-4, R-5, R-7
A-3	Build R-2/R-3/R-6 (the status engine) on the test copy, with its TEST fixture	the next build
A-4	The capture route for test evidence into the evidence folder (Final package section 3: C1 after S1–S3, or C2)	durable test evidence
A-5	Installation of tf-payguard-r1 on development, after a verified backup, at a time you choose — **not asked for now**; live is not covered by any of this	nothing until you ask
A-6	The clean database (section 6): name, starting date, master data, opening balances, logins	the last step

Not needed from you now: staff assignments (yours, later); Ravi's involvement (the 7134 pack stays prepared in the Final package, unsent); any correction to an old record.

8. The complete Payment Desk workflow: direct tests through the desk routes

Why V1 alone is not enough. The desk's Paid (`TourPaymentDeskController::paidInner`) does more than authorise: it confirms the paying account, applies the final exchange rate to a batch, calls V1 to post the ledger if the voucher is unposted, sets finalized with the bank date, writes the Paid lock (tpd_bank_approval), allocates the batch's requests and completes them, posts exchange-rate differences and bank charges, and reconciles the ledger. A failure between any two of these steps leaves the earlier ones standing; the existing retry is refused by the Paid lock (F-4). So the desk is tested as a whole, through its own routes. **[Observed: code, TourPaymentDeskController :1336–1532]**

Fixtures needed (TEST, added to the test database and listed):

Fixture	Content	Why
TEST bank account on the desk′s bank-account master, mapped to ledger 1-6-8-10380 (or a TEST ledger account)	one row, active, LKR	initiation and Paid need a paying account
Bank-charge and FX ledger mappings set to TEST accounts	desk settings	charges and FX postings must have a destination, or Paid is blocked by design
Two desk-eligible TEST payment requests on tour 990001 (one LKR supplier, one USD hotel) at status 4 with a desk voucher each	payment_request rows; vouchers at Aflag 0, created through the desk′s own issue path where possible	the single-voucher Paid and the batch cases
One bulk batch (SB-) of two TEST requests	created through bulk_create by the TEST accountant	bulk Paid
Two TEST users: an accountant (initiation) and a bank approver (Paid)	TestAccounts and TestBankAuth sign-ins	the desk′s separation rule refuses Paid by the initiator

Cases (each run from the desk′s own routes, results recorded as observed):

#	Case	Route	Must
D-1	initiate, then Paid, single LKR voucher	initiate, paid	one journal; finalized once with one bank date; one Paid lock; request complete; audit rows; reconciliation “reconciled”
D-2	Paid twice (double click, two tabs, and the second user again)	paid	second refused by the Paid lock with the voucher named; nothing written twice
D-3	bulk Paid of the SB- batch	bulk_paid	every child voucher posted once, finalized once; batch status paid; each request complete; one charge posting per tour line
D-4	bulk Paid twice; one child already Paid	bulk_paid	refused or skipped per child with a message; no second posting

#	Case	Route	Must
D-5	Paid with bank charges and a USD batch with an FX difference	paid	exactly one charge pair per line; exactly one FX entry per line; ledger reconciles to the bank debit
D-6	failure injected after step 2 (finalized set) and after step 3 (lock written); then retry	paid	today's code: retry refused, charges and FX unposted (F-4 reproduced); after the repair: either the remaining steps complete on retry or the whole Paid is undone, with an audit row either way — no half-Paid voucher
D-7	Paid by the initiator; Paid by a user without the bank-approver privilege	paid	refused (separation; permission), audited
D-8	initiate twice; initiate a finalized voucher; initiate a cancelled voucher	initiate	all refused with the reason

Dependencies. D-1 to D-8 need both TEST logins (section 7, B-1) and the fixtures above; D-6 after the repair needs the recovery design of section 9 built into the desk's Paid. Until then D-6 can only reproduce the fault. **[Proposal]**

9. Interrupted processing: the recovery design and its tests

The gap in the built guard. `AuthGuard::fail()` runs only while PHP is still running. If PHP is terminated, the connection drops, or the database stops after rows were written but before the undo finishes, nothing undoes the attempt: the claim row stays in state "posting", some ledger rows may exist, the voucher may or may not be at Aflag 1, and MySQL releases the named lock when the connection ends. A later request must not post again, and must not say "nothing was posted" when something was. **[Deduced from the design]**

Recovery design for the built MyISAM guard (designed, not built; its build is held until the storage-engine decision of section 14, because under InnoDB it reduces to the two checks of section 10c and the undo path disappears):

1. **The claim row is the record of an attempt in flight.** It is written before any posting and set to "posted" only as the last step. Any claim found in state "posting" by a later request, by the Setup page's sweep, or by a command, is an interrupted attempt and is handled before anything else happens to that voucher.

2. **Inspection under the lock.** For the voucher of the claim: the ledger rows with Recpt_No = voucher number and Dt ⩾ claimed_at are read, grouped by journal id; a journal is *complete* when its legs balance (sum of Dr = sum of Cr) and both legs exist; the voucher's Aflag and the lines' RCType are read.
3. **Decision, written to the audit before it is acted on:** (a) every journal complete, Aflag = 1, lines = 1 → the posting had finished and only the bookkeeping was lost: the claim is marked "posted (recovered)"; no row is touched. (b) anything else → the attempt is undone: the rows of that attempt are deleted by journal id, the snapshot is restored, the claim is removed; audit "failed, undone on recovery". (c) the undo itself cannot be completed → the claim is set to "needs attention", every later authorisation of that voucher is refused with "voucher N has an incomplete posting from <time>; refer to Accounts", and the Setup page lists it.
4. **Idempotent undo.** Every undo step is safe to repeat: delete by id (deleting nothing the second time), restore the snapshot values (already restored the second time), remove the claim. An interruption *during* the undo therefore leaves a "posting" claim that the next recovery finishes.
5. **Messages.** The user sees "posted", "refused: …", "failed and undone: …", or "incomplete posting, refer to Accounts" — never "nothing was posted" unless the inspection found no row of the attempt.
6. **Where it runs.** At the start of every claim for that voucher (lazy), on the desk's Setup page as a sweep over all "posting" claims older than two minutes, and as a command for IT. No scheduled job is assumed.

Controlled tests (test copy only; the shared MySQL service is never stopped):

#	Interruption	How it is produced on the test copy	Must
I-1	PHP terminates after the ledger rows were written, before the status update	test hook `pg_fail=exit_after_ledger`: PHP exits inside the posting	claim left "posting", rows present, Aflag 0; the next authorisation attempt runs recovery: undo, audit "undone on recovery", then refuses or proceeds according to the state found; tables equal to before the first attempt apart from the audit rows and the counter gap

#	Interruption	How it is produced on the test copy	Must
I-2	PHP terminates after everything was written, before the claim was marked posted	hook `pg_fail=exit_before_complete`	recovery finds complete journal + Aflag 1 → claim "posted (recovered)"; nothing deleted; a second authorisation refused
I-3	PHP terminates during the undo, after the ledger rows were deleted and before the statuses were restored	hook `pg_fail=exit_during_undo`	the next recovery finishes the undo; final state as before the attempt
I-4	the database connection drops mid-posting	hook `pg_fail=kill_connection` (`KILL CONNECTION_ID()` from a second connection, or the client closing the handle)	as I-1; MySQL has released the lock; recovery handles it
I-5	two interrupted attempts on two vouchers, then a sweep	I-1 on two TEST vouchers, then the Setup sweep	both recovered; the sweep lists both with their outcome
I-6	the database stops	**not produced**: the MySQL service is shared with development and is not stopped; argued from I-4 (an unexpected disconnect is what the application sees) and recorded as a limitation	

#	Interruption	How it is produced on the test copy	Must
I-7	the process ends immediately after COMMIT, before the outcome audit row and before the response (authorisation)	hook pg_fail=exit_after_commit (transaction-based posting of the rehearsal; on the built MyISAM guard the equivalent is an exit right after the claim row is set to posted)	the operation row is committed with its journals, the ledger lines are present and balanced, Aflag 1, RCType 1, no outcome audit row; the retry is refused as already posted and names the journal ids; it must not post again and must not say that nothing remained; the recovery or the sweep adds the outcome audit row marked recovered; counts equal to a normal success plus that one audit row
I-8	the process ends immediately after COMMIT of the desk′s Paid transaction, before the response	hook pg_fail=exit_after_paid_commit	the voucher is Paid with every posting present (lock row, allocations, FX and charge journals, reconciliation record); the retry of Paid is refused by the lock row with the voucher named; bulk Paid resumes at the next child; no second posting of any kind

Each test records: the claim row before and after, the rows of the attempt, the audit rows, the voucher and line status, and the counts and checksums of the non-TEST rows (section 12). **[Proposal]**

10. The InnoDB rehearsal package (for approval; this does not convert anything)

Your direction: evaluate InnoDB with real transactions before extending lock-claim-undo. This package is the concrete rehearsal, limited to the isolated test copy; it supersedes the earlier design note (Revised Design r3, section 9 of that document) and the cloud-side engine trial of 11 October, which ran on a reduced copy under MariaDB and is not evidence for this server (MySQL 8.4.7). Conversion of any table, anywhere, needs your separate approval after the rehearsal report; the approval wording is in section 14. [Proposal]

10a. Exact table list and dependencies

Table	Engine (observed 15 Oct, test database)	Keys now	Written by (step of the operation)	Rows deleted by
voucher	MyISAM	PRIMARY ID; UNIQUE uq_tpd_voucher_vnum (Vnum) and the BEFORE UPDATE trigger once Setup has run	creation (approval, batch_create, bulk_create, legacy save); re-rating at initiation and at Paid (Amount, g_ttl); paying account at initiation and Paid (Bank, Bank_acc); authorisation (Aflag, Aut_user, aut_date; Voucher_No is sent but dropped, see section 13); finalisation (finalized, finalized_user, finalized_date, bank_date); V5 transfer (Amount, new voucher); legacy reject (Aflag 2)	bulk undo (Bulk.php :241), V5 undo, delete_advance
v_inv	MyISAM	PRIMARY ID; no index on V_tkn	creation lines; re-rating (Amount, InvAmount, Rate); RCType at authorisation and reject; V5 (amount, new line)	batch_create failure (:988), bulk undo, V5 undo
invoice_no	MyISAM	no key; one row	counters: V_Num at every voucher creation; acc_id at every journal (one per voucher line at authorisation, one per line for FX, one per line for bank charges, one per accrual at request or approval, one per V5 transfer); voucher at every V1 call — each a read then an UPDATE without WHERE	never
account_runing	MyISAM	no primary key; non-unique index ID; the application supplies ID = acc_id, two legs share it	payment journals (V1 acc_array, Recpt_No = voucher number); FX journals (Batches::postFx, advances_pay 23); bank-charge journals (postCharges, advances_pay 24); service-fee and hotel accruals at approval or request (advances_pay 22); V5 transfer pair	guard undo (by journal id), hotel-request reject (by tour and ledger code — broader than one request), V5 undo
account_runing_2024	MyISAM	as account_runing	read only (the guard's duplicate check)	—
payment_request	MyISAM	PRIMARY id	request creation; approval status 4 or 6; reject 5; cancel 3; voucher_created flag; completion 2 (legacy route only)	never

Table	Engine (observed 15 Oct, test database)	Keys now	Written by (step of the operation)	Rows deleted by
tour_quotations	MyISAM	PRIMARY id	request flags and hotel accrual fields at request; hotel_payment_flag at authorisation and at desk Paid; reject	never
consignee	MyISAM	PRIMARY ID	request flags; transport agent at approval and issue; transport_advance and transport_settled at authorisation; completion	never
driver_allocation	MyISAM	PRIMARY id	payment_settlement at authorisation of a settlement (advances_pay 11)	never
driver_advance_request	MyISAM	PRIMARY id	request (flag 1, split row), reject (flag 0)	never
tour_activity	InnoDB	PRIMARY id	request flags for activity (lunch) requests; reject	never
daily_usd_rate	MyISAM	PRIMARY id; UNIQUE rate_date	confirm_rate; INSERT IGNORE at initiation, bulk initiation and Paid	never
tpd_request_ext	InnoDB	PRIMARY id; UNIQUE pr_id	request meta; approval figures; batch, line and voucher links; paid figures (paid_rate, paid_lkr, paid_date, fx_diff); fx_entry_id, charge_share, charge_entry_id; disputes	never
tpd_batch	InnoDB	PRIMARY id; UNIQUE batch_no, submit_token	batch and bulk creation; paying account; status; initiation figures; paid; reconciliation	bulk undo (bulk_child rows)
tpd_bank_initiation	InnoDB	PRIMARY id; UNIQUE vnum	initiation (bank reference, charges, rate, fx_diff, total_debit)	never
tpd_bank_approval	InnoDB	PRIMARY id; UNIQUE vnum	the Paid lock (desk Paid and Mark Finalized); fx_posted and fx_entries; charges_posted and charge_entries; reconciled	never
tpd_remittance	InnoDB	PRIMARY id; UNIQUE vnum (nullable since v7)	pending at Paid; prepared; sent or failed	never
tpd_document	InnoDB	PRIMARY id	bank proof at initiation; remittance; invoice and booking documents	never
tpd_expense	InnoDB	PRIMARY id	expense add and cancel	never
tpd_supplier_bank	InnoDB	PRIMARY id; UNIQUE sup_acc	supplier bank verification at approval, issue, bulk creation, confirm_supplier_bank	never
tpd_meta	InnoDB	PRIMARY k	FX and bank-charge ledger mappings (Setup page)	never
tpd_audit	InnoDB	PRIMARY id	every step, refusal and outcome (append only; a failed audit insert never stops a step)	never

Table	Engine (observed 15 Oct, test database)	Keys now	Written by (step of the operation)	Rows deleted by
pg_auth_claim	InnoDB (created on the first claim; present since G-1)	PRIMARY vnum	guard claim, posted	stale takeover; guard undo
pg_op	InnoDB (created on the first V5 transfer)	PRIMARY op_key	V5 operation key	V5 undo
tpd_bank_account	InnoDB	PRIMARY id	master-files route only; the operation reads it	—
Setup only: menu_list, user_privileges, tpd_bak_* snapshots	InnoDB; snapshots take the server default	—	the desk's Setup page, not the payment operation	—
Files, outside every transaction	—	—	writable\tpd\bank_proof (initiation upload); remittance PDFs; the mail sent by remittance_send	—
Not resolved (code not in the inspected set)	—	—	HotelRates PaymentHold and HotelCostService (hr2_* tables, at request creation and on the desk page); Mail\SendController (a mail log, if any); the dashboard2 voucher_auth route (VoucherAuthenticationController); the Voucher Cancellation screen (Aflag 3 reversal)	to be inspected before conversion

The authoritative list (one source for 10d, 10e, 12e and 14; file statements\innodb-rehearsal\CONVERSION_SET.txt, SHA-256 9b6ff261ab385d790abb67e673f006941b12ae268ff6f4b03ab1693474d51fde). Engines, keys, triggers, copies and sizes were read from information_schema of the test database at 21:55 UTC on 15 October. [Observed]

Group	Tables (exact names)	Engine now	Backup coverage (verified copies inside the test database)	Conversion scope	Restore
A — conversion set, 12 tables, about 15.0 MB	1 voucher (PRIMARY ID; UNIQUE uq_tpd_voucher_vnum; trigger tpd_voucher_vnum_guard) · 2 v_inv · 3 invoice_no (no key) · 4 account_runing (no primary key; seven non-unique indexes) · 5 account_runing_2024 (as 4) · 6 payment_request · 7 tour_quotations · 8 consignee · 9 driver_allocation (three triggers: after insert, update, delete, each inserting into 10) · 10 driver_allocation_audit (PRIMARY audit_id; written only by those triggers) · 11 driver_advance_request · 12 daily_usd_rate (UNIQUE rate_date)	MyISAM, all twelve	every one: copies of 15 Oct 12:59 for 1, 2, 3, 4, 7, 8, 9 and of 15 Oct 22:15 for 5, 6, 10, 11, 12, each equal in count and CHECKSUM TABLE EXTENDED (guard-tests\BACKUPS-20261015.json); P-2 step 1 copies all twelve again immediately before conversion	converted, one ALTER TABLE each (03_convert_innodb.sql), then the essential schema 06a	engine reversal (04) keeps later data; restoration of the pre-rehearsal state (05 + 07 triggers) returns the copies — 10d
B — copied, not converted, 20 tables	tour_activity · tpd_audit · tpd_batch · tpd_bank_account · tpd_bank_approval · tpd_bank_initiation · tpd_document · tpd_expense · tpd_meta · tpd_remittance · tpd_request_ext · tpd_supplier_bank · pg_auth_claim · pg_op (only once it exists) · hr2_tour_hotel_cost · hr2_offer_log · hr2_work_item · hr2_payment_hold · hr2_offer_choice · hr2_meta	InnoDB, all	copy of 15 Oct 12:59 for tpd_audit only; all twenty copied in P-2 step 1	not converted; copied because the test suites, the desk pages (GET) or request creation write them	RENAME TABLE from the copies if the tests changed them
C — out of scope	ci_sessions (session rows, written by every request); the 13 views (10 read account_runing); the other hr2_* tables; driver_expenses; user_acc_tbl, menu_list, user_privileges; account_posting; supplier; acc_selct (view); writable\logs\email_sents.log	—	—	not converted, not copied, not compared	—

Files: no file is changed by the conversion; writable\tpd is copied before the desk tests. Earlier counts in this plan ("eight tables" from the r3 design note, "ten tables" in the first draft of 10d, "eleven" and "thirteen tpd_ tables" in the drafts of 15 Oct before 22:40 UTC)

are superseded by this list (revision 3 of CONVERSION_SET.txt, 16 Oct, SHA-256 f28722c7821ee4e7d4effc4f05ce83dc63fc758f73a46edd1380dd55600b67ca). [Observed; Proposal for the scope]

10a-2. The four dependencies left open on 15 October, now inspected (22:10–22:30 UTC, test copy's files; line-cited inventory in the evidence folder).

Dependency	What it writes	Effect on the scope	How it is kept from affecting the rehearsal
HotelRates PaymentHold and HotelCostService (hotel payment requests; the desk pages)	PaymentHold::intercept, on a hotel request only: hr2_payment_hold (INSERT) and hr2_offer_log; markReleased: hr2_payment_hold (UPDATE). HotelCostService::computeTour, in its own transaction: DELETE + INSERT of the tour's rows in hr2_tour_hotel_cost, INSERT hr2_offer_log, then WorkQueueService::scan for the tour (inspected 16 Oct: writes hr2_work_item only; see below). computeTour is reached by a plain GET of the desk page, search_tour and payment_approval_v2 whenever a confirmed tour has no costed rows or a hotel plan changed since. No write to any Group A table; LegacyLock blocks nothing on tour_quotations.	the five hr2_ tables join Group B (copied, compared, not converted); tour 990001 is a dummy tour, so its hotel cost rows are TEST rows	the copies and the 12b comparison cover them; HotelRates\Installer is not routed and touches no Group A table (DDL on hr2_ only)
Mail\SendController (remittance and other mails)	no database write; appends writable\logs\email_sents.log on a successful send; reads the sending user's mail settings	none on the tables	the TEST users carry no mail settings, so send() returns 404 and remittance_send records the failure in tpd_remittance and tpd_audit (part of D-tests)
dashboard2 voucher_auth/update and update_tour (VoucherAuthenticationController)	voucher only: is_confirm = 1, confirm_user, confirm_date — update() on EVERY voucher with Aflag = 1 in the table, update_tour on one tour or on every non-expense authorised voucher; no ledger, no counter, no guard	a confirmation flag, not an authorisation; Group A (voucher) already covered; defect D-05 (unbounded update) added to section 13 and to the repair list	not invoked; TEST sessions are the only callers on the isolated copy; a stray call would show in the 12b comparison

Dependency	What it writes	Effect on the scope	How it is kept from affecting the rehearsal
Voucher cancellation / reversal (report/voucher_report/update, VoucherReportController)	voucher: flag = 1, Aflag = 3 by Vnum with no state check (repeatable); tour_quotations: hotel_payment_flag = 3 and pay_doller_count per night; invoice_no.acc_id once; account_runing: reads every row with Recpt_No = voucher number, swaps the ledger codes of the first two rows under the new journal number and INSERTS EVERY FURTHER ROW UNCHANGED (same journal number, same legs) — so a voucher with more than one line, or one cancelled twice, gets duplicate ledger rows; no lock, no transaction, no audit	the concrete mechanism of duplicate ledger postings found so far — recorded as D-06; Group A tables only	not invoked by any test; the route is withdrawn or re-built on the posting engine after the engine decision (business key cancel:<vnum>); its reversal journals are not part of 7134's case, which is a double authorisation posting
Routes outside the chain found on the way (not previously listed): accounts/creditor_payment/complete_payment (vouchers at Aflag 0 with ledger pairs; non-exclusive branches can post twice and the counter is consumed even when no branch matches), accounts/creditor_invoice/expenses_delete (a GET that deletes a voucher, its ledger pair and lines; its only guard can be bypassed when the voucher is already authorised), driver settlement complete_payment (ledger pair + voucher at Aflag 0, no duplicate check), driver expenses approve_expense and delete_expenses (ledger pair; a delete of a journal by id), driver fund return (ledger pair)	Group A tables	listed as D-07 and D-08 in section 13; all go to the repair list of section 3 for the posting engine	not invoked; same containment
Triggers (definitions captured 22:30 UTC into 07_triggers_20261015.sql)	tpd_voucher_vnum_guard refuses a change of Vnum (SIGNAL); the three driver_allocation triggers insert one row into driver_allocation_audit on every insert, update or delete of driver_allocation — which the settlement authorisation (payment_settlement = 1) and any undo of it perform	driver_allocation_audit joins Group A: a trigger's insert into a MyISAM table would survive a rollback	ALTER TABLE … ENGINE keeps triggers; after a RENAME-based restoration they are re-created from 07

WorkQueueService::scan, inspected on 16 October (closes the last open item of this inventory). HotelCostService::computeTour calls scan(['tour_id' => …]) after every recompute, so it runs whenever a desk page recomputes a tour. In that tour-scoped form it runs two detectors (missing rate sheets for the tour's hotels; unusual margins of the tour), both reading hr2_tour_hotel_cost, consignee, tour_quotations, hr2_hotel and the rate tables through HotelRateService's read methods (findHotelByAcc, getHotel, settings, lookup → RateLookup), and it writes exactly one table: hr2_work_item (INSERT of a new item; UPDATE of an item seen again or reopened; UPDATE to close an item no longer

found). The full scan — which also writes work_scan_at into hr2_meta — runs only without a tour id, from the work-queue page, never from a desk page. RateLookup contains no write; Settings writes hr2_meta only from its settings page and hr2_market_country only from the installer and the Hotel Rates master page (and only when that table is empty; it holds 239 rows). The other HotelRateService writers (contracts, hotels, audit) are reached from the work-queue and master-file pages, not from computeTour or scan. Result: every table a desk-page request can write is in Group A (daily_usd_rate) or Group B (tpd_ tables, tpd_audit, hr2_tour_hotel_cost, hr2_offer_log, hr2_work_item, and now hr2_meta, added as a precaution); Group B is therefore 20 tables and the copy set 32. The inspection used the 14 October development snapshot of WorkQueueService.php (SHA-256 bc0ae311…), RateLookup.php (74b1a436…) and Settings.php (8edaa23d…); the 15 October manifest comparison showed no difference between the test copy and the development copy in Libraries\HotelRates, and the test copy's own hashes are confirmed the next time the computer is reachable. Still named, not assumed: Settings::seedMarketCountries and HotelCodes::backfill (installer and master-file pages only; no route of the rehearsal reaches them), and whether a global filter protects the unfiltered POST routes of 10a-2. [Observed: code inspection 16 Oct; Proposal for the containment]

Source of the table: a line-cited inspection of the test copy's files on 15 October (TourPaymentDeskController, TourAccountsCheckController, TourAccountsController, TourPaymentDashboardController, VoucherPaymentController, BankPaymentCheckController, DriverAdvancePaymentController, ManualUpdateController, the TourPayments libraries, AuthGuard, the staged models) and the engines read from information_schema of the test database. Eleven MyISAM tables are written by the complete operation (voucher, v_inv, invoice_no, account_runing, payment_request, tour_quotations, consignee, driver_allocation, driver_allocation_audit through the triggers, driver_advance_request, daily_usd_rate) and a twelfth, account_runing_2024, is read by the guard and by the ledger reports; all twelve form the conversion set, Group A of the authoritative list below. tour_activity and every tpd_ and pg_ table are already InnoDB. Foreign keys: none involve these tables; none are added. Views: ten ledger views read account_runing. Transactions in the code today: exactly one (Vouchers::issue, used by bulk_create); every other step runs statement by statement in autocommit. Named locks today: tpd_paid_<vnum>, tpd_bulk_<id>, tpd_voucher_issue, avr_auth_<vnum>, all taken on the one shared connection, so a nested lock pair never waits on itself. About 25 other files open transactions on other tables; after conversion those transactions start to cover these tables too — a behaviour change that the regression set (10e) must exercise. Compatibility note from the file comparison of 15 October: the test copy's application files (1,001 files in the 15 Oct manifest) differ from the current development copy in 47 files — the development copy gained the AccessPolicy work (five new files; changes to LoginController, the session and permission filters, the

sidebar and several views) and a newer ManualUpdateController after the test copy was taken. Any package proven on the test copy is therefore re-applied and re-verified against the development files before an installation is proposed; that re-verification is part of A-5, not of this rehearsal. [Observed: code inspection and manifest comparison, 15 Oct]

10b. Key changes proposed with the conversion (none made yet)

Change	Essential or optional	Why
The twelve Group A tables on InnoDB	essential	without it there is no transaction over the payment tables at all
pg_operation: business_key PRIMARY KEY (one row per business operation), route, vnum, user, state, journal_ids, snapshot, claimed_at, committed_at, outcome_audit_id; inserted inside the posting transaction	essential	the primary key refuses a second posting of the same operation on any route, user or connection; the committed row is the record of truth that recovery inspects (10c)
pg_journal: journal_id PRIMARY KEY (one row per journal number), business_key, created_at; inserted inside the same transaction, one row per journal written; several rows may carry one business key	essential	two journals can never share a number; one operation may own several journals (one per voucher line today, or authorisation + charges + FX); the lines of one journal keep sharing the number in account_runing, where no unique key is placed
the journal counter taken inside the transaction by UPDATE invoice_no SET acc_id = LAST_INSERT_ID(acc_id + 1) then SELECT LAST_INSERT_ID()	essential	atomic per connection; a rolled-back take is undone with the transaction, so no counter gap from an interruption
account_runing and account_runing_2024: line_id BIGINT AUTO_INCREMENT PRIMARY KEY	optional	gives each line its own identity for later tooling; InnoDB works without an explicit key (it keeps a hidden one) and none of the guarantees depends on it
v_inv: index on V_tkn	optional	speed of the joins by voucher number only
no character-set, type or zero-date change	—	not this stage

The two essential tables are created by 06a_essential_schema.sql with the conversion, listed in Group B of the authoritative list from then on, copied and compared like the rest, and on a restoration of the pre-rehearsal state renamed to zz_conv_ copies (never

dropped). Section 11 describes the same schema from the accounting side; the earlier wording of this table ("business key UNIQUE in pg_journal") is withdrawn — the uniqueness of the business key lives in pg_operation only.

10c. Transaction boundaries (what one transaction covers)

Operation	One transaction covers (in this order)	Outside the transaction, by design	Consistency rule
Authorisation (V1, and the desk's step 1 when the voucher is unposted)	the operation row (pg_operation, business key auth:<vnum>, state posted); the atomic counter take for each journal and one pg_journal row per journal; all ledger lines of all journals of the voucher (one journal per voucher line today); voucher Aflag, Aut_user, aut_date; every line's RCType; the night flags; the transport markers (and, through the triggers, the driver_allocation_audit rows)	the named lock avr_auth_<vnum> (queueing, taken before, released after); the audit row "attempt started" written before the transaction and "posted" or "refused" after it	all or none; the business key's primary key refuses a second posting even across connections and routes; a second request waits on the lock and then meets the refusal
Desk Paid (single voucher)	the authorisation above when needed, then in the same transaction: finalized with the bank date; the Paid lock row; the remittance row (pending); the requests' final figures and the nights' flags; the batch status; the FX journals; the bank-charge journals; the reconciliation record	the named lock tpd_paid_<vnum>; the daily-rate INSERT IGNORE and the paying-account confirmation (idempotent, written first); the audit rows of the outcome	one transaction per Paid request: either the voucher is Paid with every posting present, or nothing of the Paid remains and the retry starts from the state before it — no half-Paid voucher (F-4 closed by construction)

Operation	One transaction covers (in this order)	Outside the transaction, by design	Consistency rule
Bulk Paid	one transaction per child voucher (as the row above), then one statement for the parent batch status after the last child	the named lock tpd_bulk_<id>	a failure at child k leaves children 1..k-1 Paid and k..n untouched, each recorded; the retry resumes at the first unpaid child; a child Paid twice is refused by its lock row
Voucher creation (approval, batch, bulk)	the counter take; the voucher row; its lines; voucher_created on the requests; the creation business key	the named lock tpd_voucher_issue	all or none; a double submit hits the key (bulk_create already has this shape today)
V5 transfer	the operation key; the ledger pair; the amount change on the original; the new line; the new voucher	the named lock avr_auth_<original vnum>	all or none
Request status change	the compare-and-set update and its transition row	—	one statement each; the affected-rows check decides
Initiation, remittance, documents	the row writes of the step	the file upload and the mail send, done first; the row is written only after they succeeded	a file without a row is harmless and listed by the sweep; a row without a file cannot occur

Recovery after interruption under InnoDB — the committed record decides, never the audit. The sequence of one posting is: audit row "attempt started" (autocommit, before the transaction) → BEGIN → pg_operation row, counter takes, pg_journal rows, ledger lines, statuses → COMMIT → audit row "posted" with the journal ids, and the response. An interruption can fall in three places, and only the committed state tells them apart: (1) before COMMIT — the server rolls the transaction back when the connection ends; no

pg_operation row, no pg_journal row, no line, no counter change remains; (2) after COMMIT but before the outcome audit or the response — the operation is complete and durable although no "posted" audit row and no answer were produced; (3) after the outcome audit, before the response — complete, audited, unanswered. A missing outcome audit therefore proves nothing by itself. Recovery, run lazily at the start of every attempt for the same business key and by the sweep: read pg_operation by the business key; if a row exists, read its journals (pg_journal) and the ledger lines under them, and the voucher and line statuses; when every journal of the row is present and balanced and the statuses match, the operation is committed — recovery writes the missing "posted (recovered)" audit row with the journal ids, sets outcome_audit_id, and the retry is refused as "already posted, journals …"; when the row exists but a journal is missing or unbalanced (which the transaction makes impossible unless the data was changed outside the application), the operation is marked needs_attention and every later attempt is refused with that message; when no row exists, nothing of the attempt was committed — only then is "interrupted, nothing remained" written, and the retry posts once. The answer "nothing was posted" is never derived from the absence of an audit row. The sweep also lists every "attempt started" audit row older than the lock wait with no outcome row, resolves each by the same inspection, and reports it. Files written by initiation or remittance before their row are listed by the sweep and never referenced without a row; the named locks are released by MySQL with the connection. Deadlocks: with row locks on InnoDB two Paid requests on different vouchers of one batch can lock tpd_batch rows in opposite orders; the posting code catches error 1213, rolls back and retries once, and the retry is audited. The same principle is applied to the built MyISAM guard (section 9): its claim row in state posted, read together with the ledger, is the record of truth there, not the audit. [Proposal]

10d. Backup and restore steps for the rehearsal (test copy)

1. Fresh copies inside the test database of all 32 tables of Groups A and B of the authoritative list (CREATE TABLE … LIKE, INSERT … SELECT; 01_copies.sql), each verified by count and CHECKSUM TABLE EXTENDED against its source; the before-measures of 10e (02_measures.sql with the four SHOW CREATE TRIGGER statements, 02b row hashes); a dated copy of writable\tpd; a dated archive of app\ with its SHA-256 manifest (app-before-innodb-<STAMP>.tar.gz) before the rehearsal's posting code is installed; a file dump by MySQL's own file output into writable\app_backups\ if S1–S3 show it possible, else noted as not available.
2. Conversion of the twelve Group A tables, one ALTER TABLE … ENGINE=InnoDB at a time with the session mode set to NO_ENGINE_SUBSTITUTION only (so the stored zero dates are kept as they are; the server's current sql_mode is empty), each timed (03_convert_innodb.sql); then the essential schema 06a (pg_operation, pg_journal); 06b only if your approval names it.

3. After-measures and comparison; SHOW CREATE TABLE compared (only the engine clause may differ); the thirteen views and the four triggers checked; the posting code of the rehearsal installed from its package and the test suites run (10e).
4. Two restore outcomes rehearsed, each with the measures repeated and named for what it is: (a) engine reversal — 04_rollback_alter_myisam.sql, ALTER TABLE … ENGINE=MyISAM in the same session mode; the tables return to MyISAM and keep every data change made after the conversion (TEST rows, postings, counters); triggers survive; the measures must equal those taken just before the reversal; (b) restoration of the pre-rehearsal state — 05_rollback_rename_from_copies.sql, one RENAME TABLE per Group A table (and per changed Group B table) that makes the P-2 copy the live table and keeps the converted table as a zz_conv_ copy (nothing dropped), then the four triggers re-created from 07_triggers_20261015.sql, pg_operation and pg_journal renamed to zz_conv_ copies; the measures must equal the pre-rehearsal measures. Code: unpack app-before-innodb-<STAMP>.tar.gz over app\ and compare every file with its manifest.
5. Disk: at least three times the Group A size (15.0 MB) free, plus the Group B copies.

10e. Compatibility tests

Test	Pass if
Row counts, content hashes (canonical export ordered by all columns for the keyless tables), money totals, highest ids, every counter in invoice_no, zero-date counts	equal before and after
SHOW CREATE TABLE	only the engine (and row-format wording) differs, plus the keys of 10b if approved
The ten ledger views (of the thirteen views) and the four triggers (voucher; driver_allocation after insert, update and delete)	present; same row counts and hashes
The guard tests G-1 to G-13 and the desk tests D-1 to D-8	identical outcomes; I-1 to I-4 additionally show rollback with no undo code path exercised
The regression set (desk, payment-action direct calls, E8 report-mode checks, journal posting, Bank Payment Check, Posted Expenses)	as before conversion

Test	Pass if
Existing transactions in the 25 files	each writes commit or roll back as a unit; no new deadlock in the test runs (deadlocks logged and retried once by the new code)
Rollback rehearsal	after each rollback the measures equal the before-measures

Approval this package needs: to run the rehearsal on the test copy only (copies, conversion, tests, rollback). Conversion of development or live is not part of it.

11. Journal and line keys: the proposed schema, the business key, and the shared-counter test

Today. A journal is the set of rows in account_runing that share `ID`, which is taken from the counter invoice_no.acc_id by read-then-write; the rows have no identity of their own (no primary key); nothing stops two journals from sharing an ID or one operation from being posted twice. **[Observed: M2; InvoiceNumberModel::incrementAccId]**

Proposed schema (applies under either engine; built with the InnoDB rehearsal if approved):

Element	Definition	Enforced by
Ledger line	one row of account_runing; a new column line_id BIGINT AUTO_INCREMENT PRIMARY KEY; every existing column unchanged	the primary key: every line has its own identity; nothing else about a line is unique — in particular the journal number is NOT unique per line
Journal	all lines that share ID (the journal number, kept as it is); a journal may hold any number of debit and credit lines, and they legitimately share the one ID; its legs must balance: sum(Dr) = sum(Cr)	the posting routine writes every line of a journal in one transaction and checks the balance before commit; the number is taken once per journal

Element	Definition	Enforced by
Journal number	ID, taken once per journal from invoice_no.acc_id by UPDATE invoice_no SET acc_id = LAST_INSERT_ID(acc_id + 1) then SELECT LAST_INSERT_ID() (atomic per connection, inside the transaction)	pg_journal.journal_id PRIMARY KEY: two journals can never carry the same number; the lines of one journal carry it as many times as there are lines
Business operation	the unit that may be posted once: an authorisation of one voucher (auth:<vnum>), a bank-charge posting of one voucher (charges:<vnum>), the exchange-difference posting of one voucher line (fx:<vnum>:<pr_id>), an advance transfer (transfer:<vnum>:<op_key>), an accrual at approval (accrual:<pr_id>), a settlement (settle:<alloc_id>)	pg_operation.business_key PRIMARY KEY, inserted in the same transaction as the lines: the same operation cannot post a second time, whatever route, user or connection tries; one operation may own several journals (an authorisation of a three-line voucher owns three journals today), each listed in pg_journal with its business key
Attempt record	pg_auth_claim (built) generalised to pg_operation: business key, state, snapshot, journal ids, user, times	the recovery design of section 9 (reduced under InnoDB as 10c states)

Difference from the built guard, and what is deliberately not imposed. The guard keys the claim on the voucher number; the business key above keys the operation, so one voucher can lawfully have several distinct journals (its authorisation, its bank charges, its exchange differences, a transfer) while any one of them can be posted only once. No unique key is placed on the journal number in account_runing, because the lines of one journal share it by design; uniqueness lives on pg_journal (one row per journal) and on pg_operation (one row per operation). Every legitimate ledger line of a completed operation is therefore preserved: the lines are never deduplicated by number, only the operation is refused a second time. Today's shape is kept for the rehearsal (one journal of two legs per voucher line); whether a multi-line voucher should rather post one journal with several legs is an accounting-presentation question for Ravi and is not decided here. [Proposal]

The shared-counter test (different vouchers at the same moment). A lock per voucher does not protect the counter: two authorisations of two vouchers take two different locks and both call incrementAccId, which reads acc_id and writes acc_id + 1 without a lock. Two requests can read the same value and both post journals numbered the same. **Test N-**

1: two TEST vouchers authorised at the same moment, with a test-only delay inside incrementAccId between its read and its write (hook `pg_counter_delay_ms`, honoured only with testHooks on) to widen the window; **today's code must show** two journals with one ID (the fault reproduced), or, if the window is missed, the test is repeated with the delay until the outcome is deterministic. **Repair N-2:** the atomic counter take above; **after it** the same test must show two distinct IDs every time, and the counter advanced by exactly two. **Test N-3:** twenty creations and authorisations fired together after the repair: twenty distinct voucher numbers and journal numbers, no gap beyond the recorded undone attempts. **[Proposal; N-1 is a test of today's behaviour, not an assumption]**

12. Verification expectations and the tightened failure criteria

12a. What a passing test is allowed to change. Every test names, before it runs, the TEST records it will touch (voucher ID and number, line ids, tour 990001 rows), the counters it will consume and the audit rows it will add. After the test, exactly those changes are expected and anything else is a failure of the test, whatever the response said. Expected changes of a successful authorisation (G-1 pattern): two ledger rows under one new journal number with Recpt_No = the TEST voucher number; invoice_no.acc_id advanced by one per journal written; invoice_no.voucher advanced by one (today's code consumes it on every V1 call, successful or not — section 13, D-01); the voucher's Aflag, Aut_user and aut_date; the line's RCType; a pg_auth_claim row in state posted; audit rows auth_claimed and auth_posted. Expected changes of a refusal: one audit row (auth_refused, access_denied or separation_blocked) and nothing else — not even a counter. [Proposal]

12b. Proof that unrelated records are unchanged. The TEST rows live inside the same tables as the copied data, so the comparison is made on the non-TEST part of each table against its zz_bak_20261015_1259_ copy: voucher rows with ID ⩽ 8031, v_inv rows with ID ⩽ 8252, account_runing rows with ID ⩽ 28989 or ID ⩾ 99980001 (the bank-reconciliation TEST journals already in the copy), tpd_audit rows with id ⩽ 9719, tour_quotations, consignee and driver_allocation rows other than tour 990001, and invoice_no compared with the copy plus the increments listed by the tests. Each comparison is a row count and an aggregate hash (MD5 over every column of every row in primary-key or full-column order) that must equal the copy's; a difference is listed row by row. The same comparison is run once more at the end of the suite. [Proposal]

12c. The failure criteria, tightened. After any failed or interrupted attempt — a caught exception (G-6, G-13, D-6), a terminated PHP process (I-1 to I-3), a dropped connection (I-4) — every business record of the operation (voucher, v_inv, account_runing, tour_quotations, consignee, driver_allocation, payment_request and the tpd_ rows of that operation) must be in one of exactly two states: equal to its state before the attempt, or the one complete, correct outcome of the operation (every write present, journals

balanced, links set, statuses consistent). No third state is accepted, and the response must say which of the two it is; "nothing was posted" may be answered only when the inspection found no row of the attempt. What may remain after an undone or recovered attempt, and nothing else: the audit rows of the attempt (auth_claimed, then auth_failed_undone or undone_on_recovery, each naming the journal ids removed); the attempt record in a terminal state (removed, or needs attention); counter gaps in invoice_no.acc_id equal to the journal numbers of the undone attempt, listed in the audit row; the invoice_no.voucher increment of today's V1 code; and, for an undone voucher creation, the V_Num consumed. Retry: after success the retry is refused; after an undone failure the retry succeeds exactly once; after an interruption, recovery runs first and then one of those two. In every case the number of journals per business operation ends at one or zero, never two, and no legitimate ledger line of a completed operation is lost — checked by counting journals per Recpt_No and per business key after every retry. [Proposal; applies to the built guard now and to the InnoDB design after the rehearsal]

12d. Checks run after every test. Counts of every table in 10a; values: sum of Dr = sum of Cr for every journal written, and the voucher's Amount equal to the supplier-side Dr of its payment journal; relationships: every ledger row with a Recpt_No has that voucher, every claim's journal ids exist in the ledger, every v_inv line's RCType equals its voucher's Aflag, every tpd_bank_approval row has a finalized voucher; and 12b. The evidence of each test is one file guard-tests\G-NN.json (or N-, I-, D-) with the request fields (no credentials), HTTP status, message, sent and received times, the signed-in user and user id, the TEST ids, the before and after data, the audit rows and the verdict, plus the SHA-256 of every SQL statement used to read or prepare. [Proposal]

12e. Backup coverage, checked against the authoritative list of 10a. Every Group A table now has a verified copy inside the test database: seven of 15 October 12:59 (voucher, v_inv, invoice_no, account_runing, tour_quotations, consignee, driver_allocation) and five of 15 October 22:15 (account_runing_2024, payment_request, driver_advance_request, daily_usd_rate, driver_allocation_audit), each equal in count and CHECKSUM TABLE EXTENDED (guard-tests\BACKUPS-20261015.json; the target was confirmed before the writes). The guard tests G and N and the interruption tests I change only Group A tables and the new pg_ tables; the desk tests D also write the Group B tables, which are copied before D-1 (12:59 copy exists for tpd_audit only), together with a dated copy of writable\tpd. The application files are covered by app-before-payguard-20261015-125912.tar.gz with its SHA-256 manifest, and InvoiceNumberModel.php by its pre-hook copy; any further file change on the test copy gets its own dated archive first. The trigger definitions are preserved in 07_triggers_20261015.sql. [Observed]

12f. Test hooks. Config\PayGuard.php testHooks is true on the test copy only; it enables pg_delay_ms, pg_fail and pg_counter_delay_ms (and, when built, the exit hooks of section 9). A release package carries testHooks = false and the hooks are inert; the install check

of any package compares that value. The hooks never run without the flag. [Observed: Config\PayGuard.php; Proposal for the release rule]

13. Consolidated results: every test, planned or observed

Status key: PASSED / FAILED = observed on the test copy with the evidence file named; NOT RUN = prepared, no observed result; PENDING = cannot run until the named login or build exists. Nothing in this table is inferred from code. Updated as the run proceeds. G-1′s result stands with its deviation and establishes nothing under TestAccounts; tf-payguard-r1 remains built but unproven until the suite holds under the TEST logins.

Environment re-confirmed at 21:52 UTC, 15 October, before any further write: the seven guard files and InvoiceNumberModel.php carry the hashes recorded in this section; the development copy′s VoucherPaymentController.php (e573d890…) and InvoiceNumberModel.php (3ad907c2…) are unchanged and it has no PayGuard folder; no temporary page is left in the test copy′s public folder; the test database holds voucher 7,984, v_inv 8,187, account_runing 11,394, tpd_audit 9,721 (last id 9721), one pg_auth_claim row, counters V_Num 8029 / acc_id 28991 / voucher 10008 — exactly the state after G-1 and the two fixtures; the eight copies of 12:59 are present. The browser pane′s tab-4 was found signed in again, but every session row of the test database belongs to "super admin", so the guard tests stay held until you confirm TestAccounts is the signed-in user; the user is verified again from the session rows before G-2 is sent. [Observed]

Per-session user verification (used before every test from now on). The Tour Payment Desk page renders the signed-in user of the requesting session as a constant (TPD_USER) from the server-side session; fetching that page from inside the browser tab itself, with that tab′s own cookies, therefore names that tab′s user and no other session′s. At 22:05 UTC the check on tab-4 returned "super admin" (HTTP 200, no refusal row written), so tab-4 is still the admin session and G-2 stays held. The rule for the run: a case is sent only when the tab′s TPD_USER is TestAccounts (user id 121) and a session row for that user exists; the two-user cases additionally require TestBankAuth (user id 120) verified the same way in your Chrome and two distinct session ids in ci_sessions, one per user. [Observed; Proposal for the rule]

#	Case	Status (15 Oct)	User	TEST ids	Evidence / note
G-1	normal authorisation, TEST-GUARD-1	PASSED, with a user deviation	super admin (user id 1), not TestAccounts	voucher 8032 / 8021, line 8253, journal 28991	guard-tests\G-01.json: 200 in 882 ms; journal 28991 = Dr 6-9-23-11013 1,111.11 / Cr 1-6-8-9750 1,111.11; Aflag 1, RCType 1; claim posted; audit 9720 auth_claimed, 9721 auth_posted; counts: account_runing 11,392 → 11,394, tpd_audit 9,719 → 9,721, others unchanged; counters acc_id 28990 → 28991, voucher 10007 → 10008
G-2	double click, TEST-GUARD-2	NOT RUN	TestAccounts	8033 / 8022	waits for the TestAccounts sign-in
G-3	retry after success, TEST-GUARD-1	NOT RUN	TestAccounts	8032 / 8021	
G-4	two tabs, TEST-GUARD-3	NOT RUN	TestAccounts	8034 / 8023	one user, two tabs — reported separately from G-5
G-5	two users at the same moment, TEST-GUARD-4	PENDING	TestAccounts + TestBankAuth	8035 / 8024	needs the second login in your Chrome; two tabs of one user do not satisfy it
G-6	failure after the ledger rows (hook pg_fail=after_ledger), TEST-GUARD-5	NOT RUN	TestAccounts	8036 / 8025	
G-7	retry after the undone failure	NOT RUN	TestAccounts	8036 / 8025	
G-8	reject TEST-GUARD-6, then authorise	NOT RUN	TestAccounts	8037 / 8026	
G-9	V4 bulk hotel accept	NOT RUN	TestAccounts	—	

#	Case	Status (15 Oct)	User	TEST ids	Evidence / note
G-10	V5 transfer 3,000 with an operation key	NOT RUN	TestAccounts	8038 / 8027, driver 6-9-11-99901 → 99902	
G-11	the same operation key again	NOT RUN	TestAccounts	as G-10	
G-12	V5 refusals (above balance, same driver, unauthorised voucher)	NOT RUN	TestAccounts	8038 / 8027; 8037 / 8026	
G-13	V5 failure after the lines (hook pg_fail=after_lines)	NOT RUN	TestAccounts	8038 / 8027	
G-14	desk Paid for a desk voucher	PENDING	TestAccounts + TestBankAuth	desk fixtures (section 8)	covered by D-1 once the fixtures exist
N-1	shared counter: two vouchers at the same moment (hook pg_counter_delay_ms)	PENDING	TestAccounts (+ second login)	8039 / 8028 and 8040 / 8029 (TEST-GUARD-7 and -8)	hook built 14:52 UTC; needs two signed-in sessions, because the framework's session handler serialises the requests of one login with its own named lock before the guard is reached
N-2	atomic counter take, same test	PENDING	as N-1		after N-1
N-3	twenty creations and authorisations together	PENDING	as N-1		after N-2
I-1	PHP terminates after the ledger rows	PENDING	TestAccounts	TEST voucher to be added	the exit hooks and the recovery routine are not built (held until the InnoDB decision, section 14)
I-2	PHP terminates before the claim is marked posted	PENDING	TestAccounts		as I-1
I-3	PHP terminates during the undo	PENDING	TestAccounts		as I-1
I-4	the connection drops mid-posting	PENDING	TestAccounts		as I-1

#	Case	Status (15 Oct)	User	TEST ids	Evidence / note
I-5	two interrupted attempts, then a sweep	PENDING	TestAccounts		as I-1
I-6	the database stops	NOT PRODUCIBLE	—	—	the shared MySQL service is not stopped; argued from I-4 (limitation)
D-1 to D-8	the Payment Desk workflow (section 8)	PENDING	TestAccounts + TestBankAuth	desk fixtures (section 8)	the fixtures and the desk-table copies (12e) are made once both logins are in place
I-7	process ends after COMMIT, before the outcome audit and the response (section 9)	PENDING	TestAccounts	TEST voucher to be added	needs the transaction-based posting of P-2 (or the MyISAM equivalent hook); planned, not run
I-8	process ends after COMMIT of the desk's Paid transaction, before the response	PENDING	TestAccounts + TestBankAuth	desk fixtures	as I-7; after D-1

Files and tables changed on the test copy so far (nothing on development or live):

Where	Change	Since
app\Libraries\PayGuard\AuthGuard.php (new), app\Config\PayGuard.php (new), VoucherPaymentController.php, ManualUpdateController.php, DriverAdvancePaymentController.php, voucher_print.php, transport_advance.php	tf-payguard-r1 (section 4a); SHA-256 of each file in the evidence folder	15 Oct 13:00
writable_to_delete\pg_lint_tmp.php and pg_lint_tmp2.php	the one-off compile check page, moved out of public\	15 Oct
TEST vouchers 8032–8038 (Vnum 8021–8027), lines 8253–8259, journal 28990, counters V_Num 8027 and acc_id 28990	the TEST fixtures of 4c	15 Oct 13:12
journal 28991, voucher 8032 Aflag 1, line 8253 RCType 1, pg_auth_claim (new table, one row), tpd_audit 9720–9721, counters acc_id 28991 and voucher 10008	G-1	15 Oct 14:40 UTC

Where	Change	Since
TEST vouchers 8039 and 8040 (Vnum 8028 and 8029, TEST-GUARD-7 and -8, 7,777.77 and 8,888.88, Aflag 0), lines 8260 and 8261; counter V_Num 8027 → 8029	fixtures for the shared-counter test N-1 (two unposted vouchers), copied from TEST-GUARD-2's structure; statements and SHA-256 in guard-tests\FIXTURES-20261015.json	15 Oct 14:53 UTC
app\Models\InvoiceNumberModel.php	test hook pg_counter_delay_ms inside incrementAccId(), behind Config\PayGuard testHooks (section 12f); no other change; compiles under PHP 8.1.33; SHA-256 3ad907c2… → 0be45ef7…; the unchanged file kept as writable\app_backups\InvoiceNumberModel.pre-counterhook-20261015.php	15 Oct 14:52 UTC
zz_bak_20261015_2215_account_runing_2024 (15,900 rows), _payment_request (3,110), _driver_advance_request (406), _daily_usd_rate (7), _driver_allocation_audit (1,791)	verified copies of the five Group A tables that had none (count and CHECKSUM TABLE EXTENDED equal; guard-tests\BACKUPS-20261015.json); nothing deleted	15 Oct 22:15 UTC
evidence folder: statements\innodb-rehearsal\ revision 2 (CONVERSION_SET.txt, 01–07, SHA256SUMS 6882a196…), guard-tests\BACKUPS-20261015.json	prepared statements and backup record; no database or code change	15 Oct 22:40 UTC
evidence folder: statements\innodb-rehearsal\ revision 3 (CONVERSION_SET.txt f28722c7…; 01 copies now 32 tables, hr2_meta added; 05 updated; SHA256SUMS 17d60f3d…)	WorkQueueService inventory closed (10a-2); prepared statements only; no database or code change; written to the computer when it is next reachable (the cloud evidence copy holds it meanwhile)	16 Oct 00:20 UTC

Defects and limitations observed so far (D- numbers; each goes to the repair list of section 3 or to Ravi where it is an accounting rule):

#	Observation	Kind	Consequence
D-01	V1 sends Voucher_No = the invoice_no.voucher counter to the voucher model, but Voucher_No is not in the model's allowed fields, so it is dropped silently; the counter still advances on every call (10007 → 10008 in G-1), on rejections too	Observed, G-1	a counter consumed with no stored effect; pre-existing, not introduced by the guard
D-02	the CSRF filter is commented out in app\Config\Filters.php of the test copy (and the manifest shows the same file on 15 Oct)	Observed: code	a forged cross-site POST is not refused by the framework on this copy; the permission test of section 5 must record it; the development copy has a newer Filters.php (section 10a note) to be read before any conclusion about it

#	Observation	Kind	Consequence
D-03	Config\Database.php sets DBDebug false in production, which the test copy's .env selects; raw query failures then return false instead of throwing, so several catch-based refusals in the desk code depend on explicit checks	Observed: code	the InnoDB design must enable transaction exceptions explicitly (transException) rather than rely on DBDebug
D-04	the first browser-pane session was the admin account, not TestAccounts	Observed, G-1	one test result carries a user deviation; the case is not repeated (its voucher is now authorised and serves G-3)
D-05	dashboard2 voucher_auth/update sets is_confirm, confirm_user and confirm_date on every voucher in the table that has Aflag = 1 (no tour, no id, no state check); update_tour does the same for one tour or for every non-expense authorised voucher	Observed: code (VoucherAuthenticationController :78, :110, :120)	an unbounded status change out of sequence; the route goes to the repair list (status engine, R-2/R-3) and is a candidate for the "paid items reappear" report (B-4)
D-06	voucher cancellation (report/voucher_report/update) sets Aflag = 3 by voucher number with no state check and can be repeated; it then copies every ledger row with that Recpt_No, swaps the codes of the first two under one new journal number and inserts every further row unchanged with its original journal number and legs — a voucher with more than one line, or a second cancellation, produces duplicate ledger rows; no lock, no transaction, no audit	Observed: code (VoucherReportController :408–:468)	a concrete mechanism of duplicate ledger postings (not a bank payment); repair under the posting engine with business key cancel:<vnum>; the accounting treatment of a reversal is Ravi's to confirm; whether any historical duplicate came from this route is a parked reconciliation question
D-07	accounts/creditor_invoice/expenses_delete is a GET that deletes a voucher (only while Aflag = 0), then its ledger pair and lines by journal id; because a delete that matches no row still counts as success, the ledger pair and lines of an already authorised voucher can be deleted while the voucher stays	Observed: code (CreditorInvoiceController :1086–:1088); the framework behaviour is stated, not executed	a deletion route on a GET verb with a bypassable guard; repair list (withdraw or rebuild with the state check and audit)
D-08	accounts/creditor_payment/complete_payment consumes the journal counter before any branch and its branches are not exclusive (the lunch branch is keyed on the supplier, the others on the category), so one submission can post two ledger pairs and two vouchers; driver settlement complete_payment creates a voucher and a ledger pair with no duplicate check; driver expenses delete_expenses deletes a journal by id with a guard that applies only when the client says the expense is settled	Observed: code (CreditorInvoiceController :198, :474; DriverPaymentSettlementController :160; DriverExpensesController :740)	all three go to the repair list for the posting engine (business keys per operation); none is touched before the engine decision

14. The next repair package and the precise approval it requires

What continues under the permissions already given (test copy and test database only; TEST records listed; nothing deleted): the remaining guard tests G-2 to G-13 as soon as TestAccounts is signed in on tab-4, with G-5 and G-14 pending the second login; the small test hooks needed to finish the prepared cases on today's code — pg_counter_delay_ms

in InvoiceNumberModel::incrementAccId for N-1 (built 14:52 UTC), and the exit hooks of section 9 for I-1 to I-4 — all behind testHooks and confined to the test copy; the desk fixtures and the desk-table copies of 12e; the evidence files and the update of section 13 after every case. No further lock-claim-undo code is written for any other route (your point 3). [Proposal under existing permission]

Package P-2: the InnoDB rehearsal of section 10, on the test copy only. Steps, in order, each with its measure: (1) fresh copies of all 32 tables of Groups A and B of the authoritative list, counts and checksums verified; the dated copy of writable\tpd; the archive of app\ with manifest; the before-measures of 10e including the trigger definitions; (2) ALTER TABLE … ENGINE=InnoDB for the twelve Group A tables in the stated session mode, timed, one at a time, with the after-measures and SHOW CREATE TABLE compared after each; (3) the essential schema of 10b (pg_operation, pg_journal, the atomic counter take) — part of the package; the optional keys of 10b only if you name them; (4) the rehearsal′s posting code installed from its package (one transaction per operation as 10c states, behind the same testHooks flag; the undo code stays in place but must not be exercised), then the guard tests, the counter tests and the interruption tests I-1 to I-8 on the converted copy; (5) the regression set of 10e; (6) both restore outcomes rehearsed — the engine reversal and the restoration of the pre-rehearsal state with the triggers re-created — each followed by the measures. Expected duration on the test copy: under one hour of statements; the tests take longer than the conversion. Nothing is deleted at any step. [Proposal]

Prepared for P-2, not run: the statement set statements\innodb-rehearsal\ — CONVERSION_SET.txt (revision 3), 01 copies of Groups A and B (32 tables), 02 measures with the four SHOW CREATE TRIGGER statements, 02b row-hash template, 03 conversion of the twelve tables, 04 engine reversal, 05 restoration from the copies, 06a essential schema, 06b optional keys, 07 the trigger definitions as captured — with SHA256SUMS.innodb-rehearsal.txt regenerated on 16 Oct at 00:20 UTC (SHA-256 17d60f3d…); every statement is re-hashed and run one at a time through the gated runner when you approve, with each response saved. [Prepared 15 Oct]

The approval wording this package needs (one line is enough): ″Approved: run the InnoDB rehearsal of section 10 on C:\wamp64\www\avenir_rv and avenir_restore_verify_20261011 only — fresh copies of the 32 tables of Groups A and B, conversion of the twelve Group A tables named in CONVERSION_SET.txt revision 3 (SHA-256 f28722c7…), the essential schema 06a (pg_operation, pg_journal, atomic counter take), the rehearsal′s transaction-based posting code behind testHooks, the test suites G, N, I-1 to I-8 and D, the regression set, and both restore outcomes (engine reversal; restoration from the copies with the triggers re-created); with / without the optional keys 06b; nothing deleted; no change to the development copy, the live system or any other

database." After the rehearsal report you decide R-10 (B-3), and only then is the posting design extended to the remaining routes of section 3. [Approval requested — not yet given; this plan asks for it and nothing has been converted]

Not asked for and not done under this instruction: installation on the development copy or live; creation of the clean database; any reset, import of opening balances or correction of an old record; sending Ravi's pack; staff permissions. The fresh-entry date stays unconfirmed until you specify it.